

ENTERPRISE SYSTEM DOCUMENTATION

Placement Tracker

Python FastAPI & Supabase Enterprise Edition

Prepared For: Client Handoff

Backend: Python 3.11+ / FastAPI / Uvicorn

Database: Supabase PostgreSQL (Connection Pooled)

Frontend: Responsive Glassmorphism UI

Date: October 2026

Version: 2.0.0 (Python Edition)

1. Introduction & System Architecture

The Placement Tracker application is an AI-powered enterprise portal designed to streamline job and internship discovery, resume parsing, skill verification quizzes, and automated learning gap curricula. It operates as a cohesive, single-command deployment.

Core System Components:

- **Python FastAPI Backend:** Built with Python 3.11+, FastAPI, Uvicorn ASGI server, Pydantic v2 data models, and psycpg2 threaded connection pooling. Exposes clean REST API endpoints under `/api/*` and interactive OpenAPI documentation at `/docs`.
- **Glassmorphism Frontend:** A responsive interface composed of HTML5, CSS3, and ES6 JavaScript. All pages and static assets are served directly from the `public/` directory by FastAPI, eliminating CORS issues.
- **Supabase PostgreSQL:** Acts as the relational data layer mapping students, internships, applications, and attempts. Configured with native UUID keys and automated self-bootstrapping on startup.
- **Supabase Storage:** A public storage bucket named 'resumes' safely stores student resume PDFs with secure direct links.
- **Google Gemini AI Integration:** Interacts with Gemini API models (Pro & Flash) to evaluate resumes, construct dynamic technical MCQs, and generate tailored 30-day learning curricula with direct YouTube and tutorial links. Includes an offline question bank for 15+ tech stacks.
- **PDF Text Extraction:** Utilizes pypdf for high-performance, in-memory PDF text extraction and regex matching against a master database of 160+ industry technologies.
- **SMTP Mailer Service:** Utilizes Python's smtplib with TLS encryption to dispatch password-reset verification links to students and staff.

Self-Bootstrapping Architecture: When launched against an empty Supabase database, the backend automatically creates all required tables, triggers, indexes, master skills, a default administrator account (`admin@placementtracker.com`), and starter internships. Zero manual SQL scripts required.

2. Environment Configuration

The application features an **Automatic Environment Loader**. On startup, it inspects the project root for a `.env` file, parses all parameters, and automatically decodes complex database connection URLs (including passwords with special characters like '@').

Required Keys Template:

Key	Description	Example / Default
PORT	Server listening port	3000 (or 8080)
DATABASE_URL	Full Supabase PostgreSQL connection URI	postgresql://user:pass@host:5432/postgres
DB_HOST	PostgreSQL Hostname	aws-1-ap-south-1.pooler.supabase.com
DB_PORT	PostgreSQL Connection Port	5432
DB_NAME	Database Namespace	postgres
DB_USER	Database Login Username	postgres.ftiibvtqebzlgfgedehbs
DB_PASS	Secure Database Password	your_secret_password
SUPABASE_URL	Supabase API Endpoint Link	https://ftiibvtqebzlgfgedehbs.supabase.co
SUPABASE_KEY	Service Role Key (Bypasses RLS)	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
SUPABASE_BUCKET	Bucket to upload resumes	resumes
JWT_SECRET	Min 32-character token signature key	f0141710-2155-479a-8864-7989e837b9e4
GEMINI_API_KEY	Google Gemini AI API Key	AlzaSyBhw99cLQGKdxSSvJnWRhnij2LeLDHZ79g
EMAIL_USER	SMTP Sender Email Address	your_email@gmail.com
EMAIL_PASS	SMTP App Password (16-char)	emcb xemv hjkg qjxy

Special Character URL Parsing: Supabase passwords often contain '@' or ':' symbols. The backend utilizes an intelligent reverse-split parser that isolates the hostname from credentials accurately without throwing standard URI malformed errors.

3. Database & Storage Architecture

The system uses Supabase PostgreSQL for relational persistence. When starting up, the database connection pool is initialized and tables are verified automatically.

Schema Architecture:

- **users:** Holds both student profiles and admin credentials. Uses UUID primary keys. Enforces role constraints ('student' and 'admin'). Stores skills and verified_skills as arrays.
- **skills:** Master dataset table storing over 160+ industry technologies (e.g., Python, React, Docker, Machine Learning). Used for regex resume matching and verification.
- **internships:** Stores job and internship listings including role titles, company names, stipends, modes, and required skill arrays.
- **applications:** Connects users with internships. Tracks status ('Applied', 'Shortlisted', 'Offered', 'Rejected') and feedback messages (ai_reason).
- **quiz_attempts:** Records skill verification test results, logging user ID, skill name, score (0-5), pass status, and attempted_at timestamp.

Supabase Storage Configuration:

Uploaded student resumes are stored in the 'resumes' bucket in Supabase Storage. The backend uploads PDFs and generates direct public URLs saved to the user profile:

```
INSERT INTO storage.buckets (id, name, public, file_size_limit, allowed_mime_types)
VALUES ('resumes', 'resumes', TRUE, 5242880, ARRAY['application/pdf'])
ON CONFLICT (id) DO NOTHING;
```

4. Running the Application Locally

Multiple startup mechanisms ensure frictionless execution on any machine or operating system.

Option A: Super-Smart One-Click Runner (Windows)

Double-click **run.bat** in the project root directory. The smart launcher will:

1. Check if Python 3.9+ is installed (automatically downloads and installs it if missing).
2. Create a clean virtual environment (.venv).
3. Install all required dependencies from backend/requirements.txt.
4. Automatically open your default web browser to <http://localhost:3000/index.html>.
5. Start the FastAPI server.

Option B: One-Click Runner (macOS / Linux)

Open a terminal in the root folder and run:

```
chmod +x run.sh && ./run.sh
```

Option C: Manual Python CLI

From the workspace root directory:

```
python -m venv .venv
.venv\Scripts\activate          # On Windows
source .venv/bin/activate       # On macOS/Linux
pip install -r backend/requirements.txt
python backend/run.py
```

Option D: Zero-Dependency Launch (Docker Compose)

Execute Docker Compose in the root directory:

```
docker compose up --build
```

5. Student User Guide

Step-by-step walkthrough of actions available to students on the platform:

Step 1: Account Registration & Login

Students register with their email, college details (Department, CGPA, Batch Year), and credentials. Secure JWT tokens are issued on login.

Step 2: Resume Upload & Automated Parsing

Under the Profile tab, students upload their PDF resume. The backend extracts text using pypdf, searches against 160+ master skills, and populates the skills list immediately.

Step 3: Skill Verification (Quizzes)

Students verify individual skills to earn verified badges. Clicking 'Verify' starts a 5-question technical quiz. Scoring 80% (4/5) awards a verified badge. If Gemini AI is offline, built-in fallback question banks are used.

Step 4: Job Discovery & Match Percentages

The Opportunities panel calculates real-time match ratios against student skills:

- **100% Match (Trophy):** Student possesses all required skills.
 - **Partial Match (Star/Bolt):** Student possesses some required skills.
- Clicking 'Apply Now' registers their application with status 'Applied'.

Step 5: Skill Gap Analysis

If match is below 100%, clicking 'Gap Analysis' leverages Gemini AI to generate a structured 30-day learning calendar with curated YouTube and FreeCodeCamp tutorial references.

6. Placement Officer (Admin) Guide

Key administrative tools for managing student placements, listings, and evaluations:

Step 1: Manage Job Postings

Officers post internships and full-time positions via the 'Post Job' panel, specifying company, title, stipend, mode, location, and comma-separated technical skills.

Step 2: Student Directory & CGPA Verification

Officers can browse registered students, download resumes, view verified skill badges, and toggle 'Verify College Credentials'.

Step 3: Application Review & Status Updates

Officers review candidate match scores in the 'Applications' panel. They can mark candidates as 'Shortlisted', 'Offered', or 'Rejected', and include actionable feedback saved in ai_reason.

Step 4: Leaderboard & Composite Scores

The leaderboard calculates rankings in real time using the composite formula:

Score = (CGPA * 10) + (Verified Badges * 8) + (Offered * 25) + (Shortlisted * 10) + (Apps * 2) + (College Verified ? 5 : 0)

Highlighting top performers for recruitment drives.

7. Production Deployment & Troubleshooting

Cloud Deployment Steps (Render / Railway):

1. Push codebase to GitHub.
2. Create a new Web Service on **Render.com** or **Railway.app**.
3. Set Build Command: **pip install -r backend/requirements.txt**.
4. Set Start Command: **python backend/run.py**.
5. Configure environment variables from `.env` in the dashboard settings.
6. Deploy! The static frontend auto-detects the production domain dynamically.

Interactive OpenAPI / Swagger Documentation:

Access interactive API documentation and testing directly in your browser at: **<http://localhost:3000/docs>** (Swagger UI) or **<http://localhost:3000/redoc>**.

Troubleshooting Quick Checklist:

1. **Invalid Gemini API Key:** The server automatically falls back to curated offline question banks without interrupting quizzes.
2. **Database Connection Timeout:** Verify `DB_HOST` and `DB_PORT` in `.env` match your Supabase project pooler settings.
3. **Resume PDF Extraction:** The pypdf parser processes native text streams. Scanned image PDFs will extract minimal text without OCR.
4. **Default Admin Login:** `admin@placementtracker.com` / `Admin@123` (auto-provisioned on startup if missing).